Roteiro do Vídeo de Demonstração 2

Projeto: All Invest · **Lab 7:** Serviço de Desempenho da Carteira · **Duração prevista:** cerca de 4min50s (limite: 5 min)

Integrantes:

1. Danilo Henrique de Paulo | 22.222.008-9
2. Jônatas da Silva Gonçalves | 22.123.095-6
3. Kaynã de Deus Ferreira da Silva | 22.123.096-4
4. Pedro Henrique da Fonseca do Nascimento | 22.123.099-8
5. Vinícius do Nascimento Generoso | 22.123.103-8
6. Wallace dos Santos Izidoro | 22.123.104-6

> Este roteiro foi feito para **uma pessoa** gravar sozinha. O vídeo é opcional, mas precisa mostrar o serviço em execução, as chamadas à API REST e os resultados, com pelo menos um cenário de erro. As falas são sugestões: ajuste ao seu jeito. Os valores na tela vão mudar, porque as cotações são reais, então leia o que aparecer em vez de decorar números.

Antes de gravar

1. **Código:** `git fetch`, `git checkout feature/relatorios` e `git pull`. Se a branch já tiver sido integrada, use a `develop`.
2. **Banco:** no SQL Editor do Supabase, execute `database/migrations/003_brokers_catalog.sql` e `database/migrations/003_create_incomes.sql`. Os dois podem ser executados de novo sem problema. Sem o primeiro, a lista de corretoras fica vazia.
3. **Backend:** confira se `backend/.env` tem `DATABASE_URL` e `BRAPI_TOKEN`. Depois, em um terminal:

```
cd backend
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
uvicorn src.main:app --reload --port 8000
```

4. **Segundo terminal:** abra em `backend/` com o venv ativo. Ele será usado na cena 5.
5. **Swagger:** abra `http://localhost:8000/docs`, clique em **Authorize** e preencha **ApiKeyAuth** com a `API_KEY` registrada na tabela `api_keys` (a mesma de `frontend/.env`).
6. **Dados:** faça os passos P1 a P6 do Apêndice A. Eles deixam a carteira pronta e fora do vídeo.
7. **Não exponha segredos:** nunca mostre o `.env`. O Swagger exibe um bloco **Curl** com o token e a `API_KEY` depois de cada execução. Para escondê-lo, abra o console do navegador (F12) e execute a linha abaixo; se recarregar a página, execute de novo.

```
document.head.insertAdjacentHTML('beforeend', '<style>.curl-command{display:none!important}
</style>')
```

8. **Tela:** deixe abertos o relatório `Especificacao_Implementacao_Servicos.pdf`, o VS Code com os arquivos da cena 2 e o terminal do uvicorn visível ao lado do navegador. Aumente a fonte do editor, do terminal e o zoom do navegador.

Cenas

Tempo	Seção	O que mostrar na tela	Fala sugerida
0:00 – 0:20	Abertura	README do repositório no GitHub.	Olá! Somos o grupo do All Invest, uma aplicação que centraliza investimentos em ações da B3. Neste vídeo mostramos o que implementamos no Laboratório 7: o Serviço de Desempenho da Carteira.
0:20 – 1:00	1. Especificação	Relatório, seção 1 e depois seção 3.1 (tabela de operações).	O serviço pertence ao Bounded Context Desempenho da Carteira, que apura ganhos, perdas e rentabilidade dos investimentos. Ele oferece três operações: avaliar as posições, consultar o desempenho de uma ação e obter o desempenho consolidado da carteira. Os consumidores previstos são o Report Component e o Frontend. A consulta é síncrona, por API REST com JSON, e o recálculo depois de compras e vendas é assíncrono.
1:00 – 1:40	2. Implementação	VS Code: `interfaces/components.py` (classe `IPerformanceService`), `services/performance_service.py` (`_valuate` e `_valuation`) e `routers/performance.py`.	A interface IPerformanceService traz o contrato de cada operação. O PerformanceService busca as posições no Investment Component e as cotações no Market Data, sempre pelas interfaces, e calcula o valor investido e o valor de mercado. As rotas REST ficam em routers/performance.py.
1:40 – 2:40	3. Operações da API	Swagger: passos A a D do Apêndice A. No passo A, aponte para a linha do log no terminal do uvicorn.	Agora, funcionando pelo Swagger. Já temos uma carteira com PETR4. Vou registrar uma compra de VALE3. A resposta volta na hora, e no terminal aparece o recálculo em segundo plano: essa é a comunicação assíncrona. Na rota de avaliar posições, cada ação vem com a cotação atual, o valor investido e o valor de mercado. Na consulta por ação, temos também o ganho ou perda e a rentabilidade. E no consolidado, os totais da carteira. A rentabilidade é calculada sobre os totais, não pela média das ações.
2:40 – 3:40	4. Erros do contrato	Swagger: passos E a I do Apêndice A. Destaque o código HTTP em cada resposta.	Agora os erros previstos no contrato. Uma carteira que não existe retorna 404, e o mesmo vale para a carteira de outro usuário. Um identificador inválido, como zero, retorna 422. Uma ação que não está na carteira também retorna 404. Já uma carteira sem posições não é erro: retorna uma lista vazia. E, sem autenticação, a resposta é 401.
3:40 – 4:30	5. Testes	Segundo terminal: comandos 1, 2 e 3 do Apêndice B. Destaque **passed**.	Por fim, os testes automatizados, que usam SQLite em memória e um Market Data simulado, sem depender do Supabase nem da brapi. Estes são os testes do serviço, todos aprovados. Eles cobrem também o erro 504, de tempo de consulta excedido, que é difícil de reproduzir ao vivo. E aqui a suíte completa do projeto, com 129 testes aprovados. As evidências estão em docs/evidencias/lab7.
4:30 – 4:50	Encerramento	Pasta `docs/evidencias/lab7` no GitHub.	O código, os testes, as evidências e o relatório estão no repositório do projeto. Obrigado!

Apêndice A – Passos no Swagger

Em cada rota: **Try it out**, preencher e **Execute**. Substitua `<carteira>`, `<vazia>` e `<corretora>` pelos ids anotados na preparação.

Passo	Rota	Corpo ou parâmetros	Resultado esperado
Preparação (fora da gravação)			
P1	`POST /api/users`	`{"name": "Demo Lab 7", "email": "demo.lab7@example.com", "password": "senha-demo-123"}`	201. Se a conta já existir, siga para P2.
P2	`POST /api/v1/auth/login`	`{"email": "demo.lab7@example.com", "password": "senha-demo-123"}`	200. Copie o `access_token`, clique em **Authorize** e cole em **BearerAuth**.
P3	`GET /api/brokers/catalogo`	—	200. Anote um `broker_id` como `<corretora>`.
P4	`POST /api/portfolios`	`{"portfolio_name": "Demonstracao Lab 7"}`	201. Anote o `portfolio_id` como `<carteira>`.
P5	`POST /api/portfolios`	`{"portfolio_name": "Carteira Vazia"}`	201. Anote o `portfolio_id` como `<vazia>`.
P6	`POST /api/stocks`	`{"stock_name": "PETR4", "stock_quantity": 100, "stock_price": 30.00, "broker_id": <corretora>, "trade_side": "BUY", "portfolio_id": <carteira>}`	201.
Cena 3: operações			
A	`POST /api/stocks`	`{"stock_name": "VALE3", "stock_quantity": 50, "stock_price": 60.50, "broker_id": <corretora>, "trade_side": "BUY", "portfolio_id": <carteira>}`	201. No terminal do uvicorn: `Desempenho recalculado em segundo plano.`
B	`GET /api/performance/portfolios/{portfolio_id}/positions`	`portfolio_id` = `<carteira>`	200. Lista com `current_price`, `invested_value` e `market_value`.
C	`GET /api/performance/portfolios/{portfolio_id}/positions/{stock_name}`	`portfolio_id` = `<carteira>`; `stock_name` = `PETR4`	200. Inclui `gain` e `profitability_pct`.
D	`GET /api/performance/portfolios/{portfolio_id}`	`portfolio_id` = `<carteira>`	200. Totais, `profitability_pct` e `positions`.
Cena 4: erros			
E	`GET .../{portfolio_id}/positions`	`portfolio_id` = `999999`	404 `"Carteira não encontrada."`

Passo	Rota	Corpo ou parâmetros	Resultado esperado
F	`GET .../{portfolio_id}/positions`	`portfolio_id = 0`	422 `"Input should be greater than 0"`
G	`GET .../positions/{stock_name}`	`portfolio_id = <carteira>`; `stock_name = ITUB4`	404 `"A ação ITUB4 não está nesta carteira."`
H	`GET .../{portfolio_id}/positions`	`portfolio_id = <vazia>`	200 `[]`
I	**Authorize** → **Logout** em BearerAuth → **Close**; repita o passo B	`portfolio_id = <carteira>`	401 `"Autenticação necessária."`

Apêndice B – Comandos dos testes (cena 5)

```
# 1) Testes do Serviço de Desempenho (esperado: 26 passed)
python -m pytest tests/test_performance_component.py -v

# 2) Somente os cenários de tempo excedido (esperado: 2 passed)
python -m pytest tests/test_performance_component.py -k 504 -v

# 3) Suíte completa (esperado: 129 passed)
python -m pytest -q
```

Se algo der errado

Sintoma	O que fazer

502 `"Não foi possível conectar à brapi.dev"`, mesmo com internet	Acontece em redes com inspeção de HTTPS, como no notebook do Wallace: o Python não reconhece o certificado da rede. Grave em outra rede ou pare o uvicorn e execute no mesmo terminal:
	`python -c "import os, ssl; open(os.environ['TEMP'] + '/certs.pem', 'w').writelines(ssl.DER_cert_to_PEM_cert(c) for s in ('ROOT', 'CA') for c, e, _ in ssl.enum_certificates(s) if e == 'x509_asn')"`
	`$env:SSL_CERT_FILE = "$env:TEMP\certs.pem"`
	`uvicorn src.main:app --reload --port 8000`

503 `"BRAPI_TOKEN não configurado no servidor."`	Preencha `BRAPI_TOKEN` em `backend/.env` e reinicie o uvicorn.
429 `"Limite de requisições à brapi.dev atingido."`	Aguarde alguns minutos e repita.
400 `"Corretora não cadastrada."`	Use um id de `GET /api/brokers/catalogo`. Se a lista vier vazia, execute a migração de corretoras.
401 `"Access token expirado."`	Repita o passo P2 e cole o token novo em **BearerAuth**.
409 ao criar carteira	O nome já existe. Veja o id em `GET /api/portfolios` ou use outro nome.
Passo B mostra quantidade maior que a esperada	A conta já tinha compras de ensaios. Isso não é erro, mas, para recomeçar, execute `DELETE /api/stocks/VALE3` antes do passo A.

Checklist do enunciado

Item exigido	Onde aparece
Execução do serviço implementado	Cenas 3 e 4
Chamadas às operações da API REST	Cena 3: passos B, C e D
Resultados obtidos	Cena 3: respostas em JSON; cena 5: testes
Pelo menos um cenário de erro	Cena 4: passos E, F, G e I
Duração de até 5 minutos	Cerca de 4min50s; se passar, reduza a cena 2

Depois de publicar o vídeo, coloque o link na seção 5 do relatório, no campo **Vídeo de demonstração**.